

Servidor TCP concurrente con múltiples clientes

Una empresa desea implementar un servidor para recibir solicitudes de sus clientes.

El servidor puede atender como máximo **cinco clientes conectados simultáneamente**.

Cada cliente puede permanecer conectado y realizar múltiples solicitudes durante su sesión.

Las solicitudes pueden tardar varios segundos en procesarse.

Por esta razón, el servidor deberá utilizar **hilos** para atender concurrentemente a los clientes.

Requerimientos

El servidor deberá cumplir las siguientes condiciones:

1. Utilizar comunicación **TCP**.
2. Permitir hasta **cinco clientes conectados simultáneamente**.
3. Aceptar nuevas solicitudes de conexión mientras existen clientes siendo atendidos.
4. Crear un hilo para atender a cada cliente conectado.
5. Permitir que cada cliente realice **múltiples solicitudes durante una misma conexión**.
6. Una solicitud de un cliente no deberá impedir que los demás clientes sean atendidos.
7. Mantener una estructura que contenga la información de los clientes actualmente conectados.
8. Cada conexión deberá tener información que permita identificar al cliente y al hilo que lo atiende.
9. Cuando un cliente se desconecte, deberá eliminarse correctamente de la lista de conexiones activas.
- 10.El servidor deberá verificar periódicamente que la lista de conexiones sea **íntegra**, es decir, que no contenga conexiones que ya hayan terminado y que no falte ninguna conexión que continúe activa.
- 11.El servidor deberá continuar funcionando mientras existan clientes conectados y deberá seguir aceptando nuevas conexiones cuando haya espacio disponible.

Escenario

Cada equipo deberá diseñar un protocolo sencillo para las solicitudes entre cliente y servidor.

El cliente deberá poder enviar diferentes tipos de solicitudes.

Por ejemplo:

```
PEDIDO pizza
PEDIDO cafe
PEDIDO hamburguesa
```

El servidor deberá procesar la solicitud y responder al cliente.

Cada equipo deberá definir:

- Formato de las solicitudes.
- Formato de las respuestas.
- Operaciones disponibles.
- Mensaje utilizado para finalizar la conexión.

Simulación de concurrencia

Para demostrar la concurrencia, algunas solicitudes deberán tener un **tiempo de procesamiento simulado**.

Por ejemplo:

```
Pizza          → 8 segundos
Hamburguesa    → 5 segundos
Café           → 3 segundos
```

Mientras un cliente espera que se procese su solicitud, los demás clientes deberán poder enviar nuevas solicitudes y recibir respuestas. La ejecución deberá permitir observar qué hilo está atendiendo a cada cliente.

Por ejemplo:

```
[Hilo 1] Cliente 1 conectado
[Hilo 2] Cliente 2 conectado
[Hilo 3] Cliente 3 conectado

[Hilo 1] Procesando: Pizza
[Hilo 2] Procesando: Café
[Hilo 3] Procesando: Hamburguesa

[Hilo 2] Café terminado
[Hilo 3] Hamburguesa terminada
[Hilo 1] Pizza terminada
```

El orden de finalización **no necesariamente deberá coincidir con el orden de llegada**.

Administración de conexiones

El servidor deberá mantener una lista de las conexiones activas.

Por ejemplo:

CONEXIONES ACTIVAS

Cliente	Socket	Hilo

Cliente 1	5	T1
Cliente 2	6	T2
Cliente 3	7	T3

Cada vez que un cliente se conecte, deberá incorporarse a la estructura.

Cuando un cliente termine su sesión, deberá eliminarse.

El servidor deberá disponer de un mecanismo para comprobar que la información almacenada en esta estructura sigue siendo correcta.

Escenario de prueba

Para demostrar el funcionamiento del servidor deberán realizar, como mínimo, la siguiente prueba:

1. Conectar cinco clientes.
2. Realizar múltiples solicitudes desde cada cliente.
3. Mientras los cinco clientes están conectados, intentar establecer una nueva conexión.
4. Finalizar la conexión de uno de los clientes.
5. Conectar nuevamente un cliente.
6. Continuar realizando solicitudes desde los clientes restantes.
7. Comprobar que la lista de conexiones activas se actualiza correctamente durante todo el proceso.

La ejecución deberá permitir observar claramente:

- Clientes conectados.
- Clientes desconectados.
- Hilos creados.
- Solicitudes recibidas.
- Solicitudes en procesamiento.
- Solicitudes terminadas.
- Estado actual de la lista de conexiones.

Entrega

Deberán entregar:

1. Código fuente del servidor.
2. Código fuente del cliente.
3. Evidencias de las pruebas realizadas.
4. Explicación de cómo se garantiza la concurrencia.
5. Explicación de cómo se mantiene la integridad de la lista de conexiones.

Preguntas de análisis

1. ¿Qué ocurre si un cliente permanece procesando una solicitud durante 10 segundos?
2. ¿Puede el servidor aceptar una nueva conexión mientras otro cliente está siendo atendido?
Explica cómo.
3. ¿Qué sucede cuando un cliente termina su conexión?
4. ¿Cómo detecta el servidor que una conexión ya no está activa?
5. ¿Qué problema podría ocurrir si dos hilos modifican simultáneamente la lista de conexiones?
6. ¿Qué mecanismo utilizaron para evitar inconsistencias en la lista?
7. ¿Qué ocurriría si el servidor permitiera más de cinco conexiones?
8. ¿Qué diferencia existe entre el socket utilizado para aceptar conexiones y los sockets utilizados para comunicarse con los clientes?
9. ¿Por qué cada cliente puede realizar múltiples solicitudes sin crear una nueva conexión para cada una?
10. Explica cómo se demuestra experimentalmente que el servidor realmente trabaja de manera concurrente.